

supervisor	
signature	

StudentID# : () , Name : ()

* Your answers should be written in English. To ensure readability, please use print handwriting. (No cursive writing. : 필기체 사용금지)

1. (30 points) Fill in the blanks (a)~(j) with the most appropriate English words.
 - [In CUDA] The `cudaDeviceSynchronize()` function makes host program (a. _____) until preceding tasks on the device (i.e. previously issued CUDA operations) are (b. _____).
 - [In OpenMP] By default, all variables declared outside a parallel region are (c. _____), except (d. _____) variable, which is (e. _____).
 - OpenMP consists of three key components: (f. _____), runtime library routines, and (g. _____).
 - (same as (f)): typed in your source code, these are instructions for the compiler on how to parallelize your code.
 - runtime library routines: typed in your source code, these are function calls to functions of the OpenMP library.
 - (same as (g)): handled in the terminal or OS, these are variables used by the system and/or processes.
 - [In Thrust library] The (h. _____) function applies a unary or binary operation (ex. function, functor, function pointer, etc.) to each element of an input sequence (or pair of sequences) and stores the result in an output sequence.
 - In parallel computing, (i. _____) and (j. _____) are two fundamental strategies for parallelization.
 - (same as (i)) distributes different tasks, which are concurrently performed by processes or threads, across multiple processors.
 - In (same as (j)), the same operation is performed concurrently on different subsets of the same data.

2. (8 points) [In CUDA] Following sentence describes ‘**warp divergence**’ and why it reduces parallel efficiency. Fill in the blanks with appropriate English words. Do not use the word ‘divergence’ or ‘diverge’ in your answers.

If threads in the same warp ([use less than 8 words] _____),
([use less than 8 words] _____).

3. (18 points) Following C program computes the dot product of two vectors in parallel using OpenMP and prints the result. The dot product of two vectors $\mathbf{a} = [a_1, a_2, \dots, a_n]$ and $\mathbf{b} = [b_1, b_2, \dots, b_n]$ is defined as

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n \quad , \text{ where } n \text{ is the length of the vectors.}$$

Insert appropriate program code into blanks (empty boxes) below. Your code should create M threads in an OpenMP parallel region and efficiently utilize them for the parallel computation. The vector length n is given as a command line argument.

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define M 8 // your code should create M (= 8) threads in OpenMP parallel region.

void print_vector(int* x, int len) {
    for (int i=0; i<len; i++) printf("%d ", x[i]);
    printf("\n");
}

int main (int argc, char* argv[]) {
    int i, n, result=0;
    int *a, *b;
    srand(time(NULL));
    n = atoi(argv[1]); // get the vector length from a command line argument
    a = (int*)malloc(sizeof(int)*n); // memory allocation for
    b = (int*)malloc(sizeof(int)*n); // input vectors a and b
    for (i=0; i<n; i++) {
        a[i]=rand()%10; // initialization of input vectors a and b
        b[i]=rand()%10; // by assigning random integers (less than 10)
    }

    #pragma omp parallel for
    {
        for (i=0; i<n; i++)
            result += a[i]*b[i];
    }

    print_vector(a, n); print_vector(b, n);
    printf("the dot product result: %d\n", result);
    free(a); free(b);
    return 0;
}
```

[program output example]

```
> a.out 5
```

```
0 7 1 1 6
0 5 0 3 7
the dot product result: 80
```

4. (24 points) The following CUDA program computes the transpose of a matrix and prints the result.

Transpose of a matrix is obtained by swapping the rows and columns of the matrix. For example, the transpose of $\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$ is $\begin{bmatrix} a & d & g \\ b & e & h \\ c & f & i \end{bmatrix}$.

Assume that the input matrix has an equal number (N) of rows and columns (i.e. $N \times N$ input matrix). An arbitrary integer N is given as a command line argument. The block size (= the number of threads per block) for CUDA kernel function (`transpose_dev`) is $M \times M$. In the CUDA kernel function, you must use two-dimensional thread and block indexing. Insert appropriate program code into the blanks (empty boxes) below. Your code should correctly handle matrices of arbitrary size.

<pre>#include <stdio.h> #include <stdlib.h> #include <time.h> #include <cuda_runtime.h> #define M 16 // block size (# of threads per block) is M×M void print_matrix(int* mat, int n) { for (int x=0;x<n;x++) { for (int y=0;y<n;y++) printf("%d ",mat[x+y*n]); printf("\n"); } printf("\n"); } // host function for computing transpose void transpose_host(int* in, int* out, int n) { // in:input matrix, out:output matrix, n:number of columns for (int x=0;x<n;x++) { for (int y=0;y<n;y++) { out[x+y*n]=in[y+x*n]; } } } // CUDA kernel (device) function for computing transpose __global__ void transpose_dev(int* in, int* out, int n) { // in:input matrix, out:output matrix, n:number of columns</pre>	<pre>int main (int argc, char* argv[]) { int N = atoi(argv[1]); // N is the number of columns in matrix int *in, *out_host, *out_device; // declare matrix variables srand(time(NULL)); // unified memory allocation for in, out_host, and out_device for (int x=0;x<N;x++) // input matrix initialization for (int y=0;y<N;y++) in[x+y*N]=rand()%10; transpose_host(in,out_host,N); transpose_dev (in,out_device,N); cudaDeviceSynchronize(); print_matrix(in,N); // print input matrix print_matrix(out_host,N); // print output matrix (host) print_matrix(out_device,N); // print output matrix (device) cudaFree(in); cudaFree(out_host); cudaFree(out_device); return 0; }</pre>
	<u>[program output example]</u> <pre>> a.out 3 6 2 5 3 8 9 8 3 4 6 3 8 2 8 3 5 9 4 6 3 8 2 8 3 5 9 4</pre>

5. (20 points) (a) In prefix sum algorithm, input is a sequence of n elements $\{x_1, x_2, \dots, x_n\}$ with a binary addition operation denoted by $\oplus$, and output is $\{s_1, s_2, \dots, s_n\}$, where $s_i =$ for $1 \leq i \leq n$. Fill in the blank.

(b) Fill in the blanks with suitable pseudocode for the parallel prefix sum algorithm implemented using recursion.

```
ParallelPrefixSum ( $\langle x_1, x_2, \dots, x_n \rangle, \oplus$ )
// assume  $n = 2^k$  for some  $k \geq 0$ , return prefix sum  $\langle s_1, s_2, \dots, s_n \rangle$ 
if n=1 then
     $s_1 \leftarrow x_1$ ;
else {
    parallel for  do
        
    parallel for  do
        if i=1 then  $s_1 \leftarrow x_1$ 
        else if  then 
        else 
    }
return  $\langle s_1, s_2, \dots, s_n \rangle$ ;
```